

# LUMEN-INDEX-INTEGRITY

## Lumen Index Integrity Report

**Subject:** Did the ollama NaN embedding fault corrupt the Lumen semantic index for this project?

**Database:** ~/.local/share/lumen/21bf1507a8925bcf/index.db (101,474,304 bytes) **Project:** <ext-volume-host>/Projects/vasic  
**Investigated:** 2026-08-26, 23:19–23:31 (+02:00) **Method:** read-only forensic inspection. Every connection was opened as `sqlite3.connect("file:<home>/.local/share/lumen/21bf1507a8925bcf/index.db?mode=ro", uri=True)` with `PRAGMA busy_timeout`. No write was ever issued; no `lumen subcommand` was run. A concurrent `lumen index` process was writing to the DB throughout.

---

### VERDICT

The index is NOT CORRUPT. It is TRUSTWORTHY BUT SEVERELY INCOMPLETE.

Do NOT rebuild it. Rebuilding would throw away 30,009 verified-good vectors and restart a ~15-hour job from zero. The correct action is to **let it finish** (or re-run `lumen index`), now that the ollama backend is healthy again.

| Question                     | Answer                                                      |
|------------------------------|-------------------------------------------------------------|
| Any NaN / Inf vectors?       | 0 — out of 30,009 decoded (100% of the index, not a sample) |
| Any all-zero vectors?        | 0                                                           |
| Any wrong-dimension vectors? | 0                                                           |
| Any zero / absurd L2 norms?  | 0                                                           |

| Question                           | Answer                                             |
|------------------------------------|----------------------------------------------------|
| Any null or empty embedding blobs? | 0                                                  |
| PRAGMA integrity_check             | ok                                                 |
| Coverage                           | 491 of 3,833 indexable files fully indexed (12.8%) |

**Why no corruption was possible:** when ollama returned HTTP 500, Lumen’s embedding client raised an error and the *entire indexing run aborted*. It never received a vector, so it never had anything to write. There is no code path that substitutes a placeholder. This is confirmed by the log (see §4) and by the data (see §2).

**The real damage is coverage, not corruption.** The failures aborted seven consecutive runs, so the index only ever got through the alphabetically-first ~13% of the tree. Semantic search currently returns **nothing** for \_tests/, \_tools/, design-system/, docs/, scripts/, \_content\_sr/, \_content\_tr/, \_content\_zh/ and others (see §5).

## 1. Schema

Read with:

```
SELECT type, name, tbl_name, sql FROM sqlite_master ORDER BY
    type, name;
PRAGMA table_info('<table>');
```

### Application tables

```
CREATE TABLE files (
    path TEXT PRIMARY KEY,
    hash TEXT NOT NULL
);

CREATE TABLE chunks (
    id TEXT PRIMARY KEY,
    file_path TEXT NOT NULL REFERENCES files(path),
    symbol TEXT NOT NULL,
    kind TEXT NOT NULL,
    start_line INTEGER NOT NULL,
    end_line INTEGER NOT NULL
);
CREATE INDEX idx_chunks_file_path ON chunks(file_path);

CREATE TABLE project_meta (
```

```

    key TEXT PRIMARY KEY,
    value TEXT NOT NULL
);

```

## Vector table

```

CREATE VIRTUAL TABLE vec_chunks USING vec0(
    id TEXT PRIMARY KEY,
    embedding float[768] distance_metric=cosine
);

```

This is **sqlite-vec vec0**, created by extension version **v0.1.6** (SELECT key, value FROM vec\_chunks\_info; → CREATE\_VERSION = 'v0.1.6').

The vec0 module is not loadable from stock CPython, so SELECT ... FROM vec\_chunks is impossible in a plain sqlite3 connection:

```
sqlite3.OperationalError: no such module: vec0
```

**I therefore decoded the shadow tables directly**, which vec0 maintains as ordinary SQLite tables:

```

CREATE TABLE "vec_chunks_rowids"(
    rowid INTEGER PRIMARY KEY AUTOINCREMENT,
    id TEXT UNIQUE NOT NULL,      -- == chunks.id
    chunk_id INTEGER,             -- which block
    chunk_offset INTEGER          -- slot within the block
);
CREATE TABLE "vec_chunks_chunks"(
    chunk_id INTEGER PRIMARY KEY AUTOINCREMENT,
    size INTEGER NOT NULL,        -- slots per block (1024 here)
    validity BLOB NOT NULL,       -- size-bit LSB-first bitmap
    rowids BLOB NOT NULL          -- size x int64
);
CREATE TABLE "vec_chunks_vector_chunks00"(
    rowid PRIMARY KEY,           -- == chunk_id
    vectors BLOB NOT NULL        -- size x 768 x float32
);

```

## How embeddings are actually stored — confirmed by measurement

| Property | Value                                                    | Evidence                                                            |
|----------|----------------------------------------------------------|---------------------------------------------------------------------|
| Encoding | IEEE-754<br><b>float32, little-endian,</b><br>contiguous | numpy.frombuffer(blob,<br>dtype='<f4') yields unit-<br>norm vectors |

| Property         | Value                                                              | Evidence                                                             |
|------------------|--------------------------------------------------------------------|----------------------------------------------------------------------|
| Dimensionality   | <b>768</b>                                                         | declared in DDL;<br>project_meta.vec_dimensions<br>= 768             |
| Bytes per vector | <b>3,072</b>                                                       | 768 x 4                                                              |
| Block layout     | <b>1,024 vectors<br/>per blob</b>                                  | length(vectors) = 3,145,728<br>= 1024 x 768 x 4 for all 30<br>blocks |
| Validity bitmap  | 128 bytes =<br>1,024 bits                                          | length(validity) = 128                                               |
| Rowid sidecar    | 8,192 bytes =<br>1,024 x int64                                     | length(rowids) = 8192                                                |
| Normalization    | vectors are <b>L2-<br/>normalized to<br/>1.0 at write<br/>time</b> | see §2 norm distribution                                             |

Verified with:

```
SELECT c.chunk_id, c.size, length(c.validity), length(c.rowids),
       length(v.vectors)
FROM   vec_chunks_chunks c
LEFT JOIN vec_chunks_vector_chunks00 v ON v.rowid = c.chunk_id
ORDER BY c.chunk_id;
```

All 30 blocks returned size=1024, 128, 8192, 3145728. Blocks 1–29 are fully occupied (1024 live slots each); block 30 held 49 live slots at snapshot 1 and 313 at snapshot 2.

## project\_meta — confirms this is the right database

```
SELECT key, value FROM project_meta ORDER BY key;
```

| key             | snapshot 1 (23:21:21)                  | snapshot 2<br>(23:27:47) |
|-----------------|----------------------------------------|--------------------------|
| project_path    | <ext-volume-host>/<br>Projects/vasic   | (same)                   |
| embedding_model | ordis/jina-embeddings-v2-<br>base-code | (same)                   |
| vec_dimensions  | 768                                    | (same)                   |
| total_files     | 3835                                   | 3833                     |
| root_hash       | e762a114...ab096                       | 8f46dd1d...5980be        |

| key             | snapshot 1 (23:21:21) | snapshot 2 (23:27:47) |
|-----------------|-----------------------|-----------------------|
| last_indexed_at | 2026-08-26T21:02:37Z  | 2026-08-26T21:25:19Z  |

project\_path matches exactly. **This is the correct index for this project.**

## 2. Are any stored vectors corrupt?

**No. Zero corrupt vectors.** I decoded **every live vector**, twice — not a sample. (The brief called for  $\geq 200$  sampled or all if under 2000; there are  $\sim 30,000$ , and a full scan was cheap, so all of them were checked in both passes.)

### Procedure

```
-- 1. every live slot and which chunk it belongs to
SELECT id, chunk_id, chunk_offset FROM vec_chunks_rowids;

-- 2. chunk -> file mapping, for naming any bad row
SELECT id, file_path, symbol, kind, start_line, end_line FROM
    chunks;

-- 3. the raw vector blocks
SELECT c.chunk_id, c.size, c.validity, c.rowids, v.vectors
FROM vec_chunks_chunks c
JOIN vec_chunks_vector_chunks00 v ON v.rowid = c.chunk_id
ORDER BY c.chunk_id;
```

Each block was decoded as `np.frombuffer(vectors, dtype='<f4').reshape(size, 768)`, the validity bitmap as `np.unpackbits(..., bitorder='little')[:size]`, and every slot present in `vec_chunks_rowids` was tested for `np.isnan`, `np.isinf`, all-zero, dimension, and `np.linalg.norm` in float64.

### Results

| Check                  | Snapshot 1 (29,745 vectors) | Snapshot 2 (30,009 vectors) |
|------------------------|-----------------------------|-----------------------------|
| NaN-containing vectors | 0                           | 0                           |
| Inf-containing vectors | 0                           | 0                           |
| All-zero vectors       | 0                           | 0                           |

| Check                                     | Snapshot 1<br>(29,745 vectors) | Snapshot 2<br>(30,009 vectors) |
|-------------------------------------------|--------------------------------|--------------------------------|
| Wrong dimensionality (!= 768)             | 0                              | 0                              |
| L2 norm == 0 or absurd (<1e-6 or >1e6)    | 0                              | 0                              |
| Validity-bitmap vs rowid-table mismatches | 0                              | 0                              |

**No bad vector exists, so there is no affected file/chunk to name.**

## L2 norm distribution (snapshot 2, n = 30,009)

```

min      = 0.999999      median = 1.000000      max   = 1.000001
p0.1     = 0.999999      mean   = 1.000000      std   = 0.000000
p1       = 1.000000      p99    = 1.000000
count norm < 0.5 : 0      count norm > 100 : 0

```

Every vector is a unit vector to float32 precision. This is significant: **NaN is absorbing under normalization**. If even one component of a vector had been NaN, its norm would have been NaN and the whole vector would have been written as NaN. A perfectly tight distribution at 1.0 across all 30,009 vectors is direct proof that no NaN ever reached the writer.

## Distinctness (guards against a repeated garbage/placeholder vector)

4,096 vectors from blocks 1–4 were MD5-hashed: **4,096 distinct, 0 byte-identical groups, max group size 1**. There is no repeated constant/placeholder vector.

## Independent proof that the decoding is correct

The “all norms are 1.0” result would be meaningless if I had mis-read the byte layout. To rule that out, I re-embedded the original source text of three chunks with the *live* ollama backend and compared with the stored vector:

| Chunk            | File:lines                                     | cosine vs re-embedded<br>path\nsymbol\nbody | control<br>(vs<br>unrelated<br>stored<br>vector) |
|------------------|------------------------------------------------|---------------------------------------------|--------------------------------------------------|
| 007fc48c9f28f674 | _content/products/<br>HelixTerminator.md:87-93 | 0.9796                                      | 0.5896                                           |
| 02e1e07d54bf256d | _content/products/<br>HelixBuilder.md:31-38    | 0.9922                                      | 0.5133                                           |
| 039bb9ab95ce8912 | _content/products/<br>LLMOrchestrator.md:72-82 | 0.9858                                      | —                                                |

Cosine ~0.98–0.99 against a fresh embedding of the same source text, versus ~0.51–0.59 for unrelated content. The byte layout is decoded correctly **and** the stored vectors are semantically valid — they still match what the model produces today.

### 3. Did Lumen store anything at all when the backend was failing?

No. Nothing. Not a single row, and certainly no placeholder vectors.

#### Structural evidence — the index is perfectly self-consistent

```

SELECT COUNT(*) FROM chunks c LEFT JOIN vec_chunks_rowids r ON
    r.id = c.id WHERE r.id IS NULL;    -- 0
SELECT COUNT(*) FROM vec_chunks_rowids r LEFT JOIN chunks c ON
    c.id = r.id WHERE c.id IS NULL;    -- 0
SELECT COUNT(*) FROM vec_chunks_rowids WHERE chunk_id IS NULL OR
    chunk_offset IS NULL;              -- 0
SELECT COUNT(*) FROM chunks c LEFT JOIN files f ON f.path =
    c.file_path WHERE f.path IS NULL;  -- 0
SELECT COUNT(*) FROM chunks WHERE end_line <
    start_line;                        -- 0
SELECT COUNT(*) FROM chunks WHERE start_line <
    0;                                 -- 0
SELECT COUNT(*) FROM chunks WHERE symbol =
    '';                                --
0

```

chunks and vec\_chunks\_rowids are exactly 1:1 (29,745 = 29,745; then 30,009 = 30,009). **There is no chunk without a vector and no vector without a chunk.** There is no room for a “row written with a missing embedding” — the schema pairs them and the data honours it.

There are no NULL or empty embedding blobs: the vectors column is BLOB NOT NULL and all 30 blocks are exactly the full 3,145,728 bytes.

**Lumen commits incrementally per batch, and each commit is valid**

This is provable from the run log, ~/.local/share/lumen/debug.log. Each run reports how many files the *previous* run had already banked:

| Run started         | files_unchanged at plan time | Outcome                             |
|---------------------|------------------------------|-------------------------------------|
| 2026-08-24 12:13:34 | 0                            | FAILED (NaN, batch 192) after 1h46m |
| 2026-08-26 20:38:45 | 193                          | FAILED (HTTP 400, EOF, batch 96)    |
| 2026-08-26 22:34:23 | 412                          | FAILED (NaN, batch 224)             |
| 2026-08-26 23:03:46 | 448                          | FAILED (NaN, batch 0)               |
| 2026-08-26 23:04:29 | 448                          | FAILED (NaN, batch 0)               |
| 2026-08-26 23:06:45 | 448                          | FAILED (NaN, batch 32)              |
| 2026-08-26 23:11:21 | 448                          | cancelled by signal at 23:25:19     |
| 2026-08-26 23:25:50 | 491                          | running                             |

The files\_unchanged count climbs 0 → 193 → 412 → 448 → 491 **across runs that all failed.** That is the signature of per-batch commits: an aborted run leaves behind a *partial but valid* index, and the next run picks up where it left off. Exactly the benign failure mode.



Note the three runs at 23:03, 23:04 and 23:06 that died at **batch 0** / **batch 32** — they contributed nothing at all (`files_unchanged` stays pinned at 448). When the backend was fully wedged, Lumen wrote **zero** rows rather than garbage ones.

## The one thing that *looks* alarming, and why it is not

At snapshot 1, **164 rows in files had an empty hash and zero chunks**:

```
SELECT COUNT(*) FROM files WHERE hash =  
    '';  
164 -> 175  
SELECT f.path FROM files f LEFT JOIN chunks c ON c.file_path =  
    f.path  
WHERE c.id IS NULL ORDER BY  
    f.path;  
-- same set
```

All 164 are non-empty `.md` files on disk (3,773–29,616 bytes; **0 missing, 0 zero-byte**), so they *should* have produced chunks. These are **placeholder files rows written before the embedding step**, left behind when a run aborted.

Comparing the two snapshots settles what they are:

|                                                                      | count                                                                  |
|----------------------------------------------------------------------|------------------------------------------------------------------------|
| Empty-hash rows at snapshot 1                                        | 164                                                                    |
| Empty-hash rows at snapshot 2                                        | 175                                                                    |
| <b>Resolved</b> (empty hash → real hash + chunks) between snapshots  | <b>23</b> (all <code>_content_ko/...</code> )                          |
| <b>Newly created</b> placeholders (the live frontier)                | <b>34</b> (all <code>_content_ru/</code> , <code>_content_sr/</code> ) |
| <b>Stuck in both snapshots</b> (leftovers from earlier aborted runs) | <b>141</b>                                                             |

That “141” is not a coincidence: the 23:11 run’s plan reported **files\_to\_modify: 141**. Lumen already classifies these rows as needing reprocessing, because an empty hash can never match a real content hash.

**These placeholder rows are harmless and self-healing:**

- they carry **zero chunks and zero vectors** — there is no garbage to poison a search;

- the empty hash guarantees they are re-processed on the next run;
- they demonstrably resolve on their own (23 of them did so during the 6 minutes between my two snapshots).

A breakdown of the 141 stuck placeholders, and the full list, is in §6.

## Lumen's error handling, from the binary

Strings extracted from lumen-linux-amd64 v0.0.41 show the failure path is a hard error, never a fallback value:

```
service returned HTTP %d
embedding server failed, trying next
all embedding servers exhausted
all embedding servers are unhealthy
no healthy embedding server found
```

and the write path is:

```
INSERT INTO files (path, hash) VALUES (?, ?)
DELETE FROM chunks WHERE file_path = ?
```

There is no “insert zero vector on error” path. The observed log errors are all of the form indexing: embed batch: all embedding servers exhausted after failover: ... — the batch throws, the run dies.

---

## 4. The fault itself, dated from the log

Source: ~/.local/share/lumen/debug.log (7,225,413 bytes; covers 2026-05-29 → 2026-08-26).

```
grep -c "unsupported value: NaN" ~/.local/share/lumen/
debug.log      # 140
grep -c "all embedding servers exhausted" ~/.local/share/lumen/
debug.log      # 143
```

- **140** log entries carry {"error": "failed to encode response: json: unsupported value: NaN"}.
- **First occurrence: 2026-07-22T20:59:52+03:00. Last: 2026-08-26T23:11:13+02:00.** The fault window is therefore **~5 weeks**, not a single evening.
- Every one of the 140 is at level ERROR with msg of indexing failed (65) or indexing nested repo failed (75). **In all 140 cases the run aborted.** Not one is a warning-and-continue.

- It affected many projects (boba 19, claude\_toolkit 16, lava 13, tmux 6, ...).

### Five of those hits are this project:

```
2026-08-24T13:59:12 embedding batch starting at 192 (HTTP 500)
NaN
2026-08-26T23:02:37 embedding batch starting at 224 (HTTP 500)
NaN
2026-08-26T23:04:29 embedding batch starting at 0 (HTTP 500)
NaN
2026-08-26T23:05:31 embedding batch starting at 0 (HTTP 500)
NaN
2026-08-26T23:11:13 embedding batch starting at 32 (HTTP 500)
NaN
```

Plus one unrelated backend failure on 2026-08-26T22:34:17: embed error (HTTP 400): {"error":"do embedding request: Post \"http://127.0.0.1:46279/embedding\": EOF"} (the ollama model runner died mid-request).

### This project's index has never once completed

```
grep -c '"msg":"indexing complete"' ~/.local/share/lumen/
debug.log # 39 (all other projects)
```

Filtering the log to "project":"<ext-volume-host>/Projects/vasic" yields **7 indexing started, 6 indexing failed, and 0 successful indexing complete**. The only indexing complete for this project is the 23:25:19 cancellation record, which reports indexed\_files: 0, chunks\_created: 0, elapsed: 13m56s and an empty new\_root\_hash.

### Backend health now

```
GET http://localhost:11434/ -> HTTP 200
POST http://localhost:11434/api/embed -> HTTP 200, 768 finite floats, no NaN
```

**ollama is healthy again as of 2026-08-26 23:26.**

---

## 5. Coverage vs the working tree — two snapshots

A lumen index <ext-volume-host>/Projects/vasic process was writing to the DB throughout. Both snapshots were taken read-only, in place.

### Snapshot deltas

| Metric                                                            | Snapshot 1 —<br>2026-08-26T23:21:21+02:00 | Snapshot 2 —<br>2026-08-26T23:27:47+02:00 |
|-------------------------------------------------------------------|-------------------------------------------|-------------------------------------------|
| SELECT COUNT(*) FROM files                                        | 632                                       | 666                                       |
| SELECT COUNT(*) FROM chunks                                       | 29,745                                    | 30,009                                    |
| SELECT COUNT(*) FROM<br>vec_chunks_rowids                         | 29,745                                    | 30,009                                    |
| SELECT COUNT(*) FROM files<br>WHERE hash <> '' (fully<br>indexed) | 468                                       | 491                                       |
| SELECT COUNT(*) FROM files<br>WHERE hash = ''<br>(placeholders)   | 164                                       | 175                                       |
| Live vectors in block 30                                          | 49                                        | 313                                       |
| project_meta.last_indexed_at                                      | 2026-08-26T21:02:37Z                      | 2026-08-26T21:25:19Z                      |

**The index is demonstrably moving forward, and every vector added between the two snapshots was also clean** (the full-decode scan was re-run at snapshot 2 and again found 0 defects across all 30,009).

Progress rate: ~23 fully-indexed files per 6.4 minutes  $\approx$  **3.6 files/min.**  
At that rate the remaining 3,342 files need roughly **15 hours**.

### Working tree

```
git ls-files | wc -l # 5679
               tracked files
git ls-files | grep -v -E '\.(png|jpg|...|woff2|ttf|pdf|zip)$' |
wc -l # 4473 non-binary
```

Lumen’s own count of indexable files (from its plan log) is **3,833**.

|                           | count |
|---------------------------|-------|
| Indexable files per Lumen | 3,833 |

|                                                | count                |
|------------------------------------------------|----------------------|
| Rows present in files                          | 666 (17.4%)          |
| <b>Fully indexed (hash + chunks + vectors)</b> | <b>491 (12.8%)</b>   |
| Placeholder rows, no vectors yet               | 175                  |
| <b>Not reached at all</b>                      | <b>3,167 (82.6%)</b> |

## Per-directory coverage (snapshot 2)

```
SELECT CASE WHEN instr(path, '/') > 0 THEN
    substr(path, 1, instr(path, '/') - 1) ELSE '<root file>' END
    AS top,
    COUNT(*)
FROM files GROUP BY top ORDER BY 2 DESC;
```

| Top-level dir | indexable in tree | rows in index | fully indexed |
|---------------|-------------------|---------------|---------------|
| _content      | 69                | 69            | 69            |
| _content_ar   | 38                | 38            | 38            |
| _content_de   | 38                | 38            | 38            |
| _content_es   | 38                | 38            | 38            |
| _analysis     | 93                | 73            | 73            |
| .specify      | 79                | 67            | 67            |
| _content_fa   | 38                | 38            | 33            |
| _content_be   | 38                | 38            | 24            |
| _content_fr   | 38                | 38            | 22            |
| _content_ko   | 38                | 38            | 20            |
| _content_hi   | 38                | 38            | 14            |
| .claude       | 15                | 10            | 10            |
| .kimi-code    | 15                | 10            | 10            |
| .opencode     | 15                | 10            | 10            |
| .github       | 1                 | 1             | 1             |
| <root file>   | 8                 | 5             | 1             |
| _content_ja   | 38                | 38            | 0             |
| _content_kk   | 38                | 38            | 0             |
| _content_ru   | 38                | 38            | 0             |
| _content_sr   | 38                | 3             | 0             |

Top-level directories ENTIRELY ABSENT from the index (12)

| Directory     | Indexable files invisible to search     |
|---------------|-----------------------------------------|
| _tests        | 3,406                                   |
| design-system | 136                                     |
| _tools        | 94                                      |
| _content_tr   | 38                                      |
| _content_zh   | 38                                      |
| _content_sr   | 38 (only 3 placeholder rows, 0 indexed) |
| scripts       | 4                                       |
| submodules    | 2                                       |
| .codegraph    | 1                                       |
| data          | 1                                       |
| docs          | 1                                       |
| upstreams     | 1                                       |

The pattern is a strict ASCII-sorted sweep — .claude, .github, .kimi-code, .opencode, .specify, then \_analysis, \_content\*... — currently stalled at \_content\_ru / \_content\_sr. Everything sorting after that has never been reached. **Note that docs/ — including this report’s own directory — is not indexed.**

6. The 141 stuck placeholder rows

Rows in files with hash = '' and zero chunks in both snapshots. They hold **no vectors**, so they cannot corrupt a search result; they will be re-processed automatically because an empty hash never matches a real content hash. Listed for completeness.

|    |             |                                            |
|----|-------------|--------------------------------------------|
| 4  | <root file> | (AGENTS.md, CLAUDE.md, GEMINI.md, QWEN.md) |
| 14 | _content_be |                                            |
| 5  | _content_fa |                                            |
| 16 | _content_fr |                                            |
| 24 | _content_hi |                                            |
| 38 | _content_ja |                                            |
| 38 | _content_kk |                                            |
| 2  | _content_ko |                                            |

Full list:

AGENTS.md  
CLAUDE.md  
GEMINI.md  
QWEN.md  
\_content\_be/products/HelixConstitution.md  
\_content\_be/products/HelixGitpx.md  
\_content\_be/products/HelixLLM.md  
\_content\_be/products/HelixMemory.md  
\_content\_be/products/HelixOTA.md  
\_content\_be/products/HelixPlay.md  
\_content\_be/products/HelixQA.md  
\_content\_be/products/HelixSkills.md  
\_content\_be/products/HelixSpecifier.md  
\_content\_be/products/HelixTerminator.md  
\_content\_be/products/HelixTrack.md  
\_content\_be/products/HelixTranslate.md  
\_content\_be/products/Herald.md  
\_content\_be/products/LLMOrchestrator.md  
\_content\_fa/products/VisionEngine.md  
\_content\_fa/products/docs\_chain.md  
\_content\_fa/products/task\_bridge.md  
\_content\_fa/sites/milosvasic-ru.md  
\_content\_fa/sites/vasic-digital.md  
\_content\_fr/docs/cover-letter.md  
\_content\_fr/docs/cv.md  
\_content\_fr/docs/portfolio.md  
\_content\_fr/products/Catalogizer.md  
\_content\_fr/products/Courses-Creator.md  
\_content\_fr/products/DocProcessor.md  
\_content\_fr/products/Helix-Flow.md  
\_content\_fr/products/HelixAgent.md  
\_content\_fr/products/HelixBuilder.md  
\_content\_fr/products/HelixCluster.md  
\_content\_fr/products/HelixCode.md  
\_content\_fr/products/HelixConstitution.md  
\_content\_fr/products/HelixGitpx.md  
\_content\_fr/products/HelixLLM.md  
\_content\_fr/products/HelixMemory.md  
\_content\_fr/products/HelixOTA.md  
\_content\_hi/products/HelixMemory.md  
\_content\_hi/products/HelixOTA.md  
\_content\_hi/products/HelixPlay.md  
\_content\_hi/products/HelixQA.md  
\_content\_hi/products/HelixSkills.md  
\_content\_hi/products/HelixSpecifier.md  
\_content\_hi/products/HelixTerminator.md  
\_content\_hi/products/HelixTrack.md  
\_content\_hi/products/HelixTranslate.md  
\_content\_hi/products/Herald.md  
\_content\_hi/products/LLMOrchestrator.md  
\_content\_hi/products/LLMProvider.md  
\_content\_hi/products/LLMsVerifier.md

\_content\_hi/products/Mail-Server-Factory.md  
\_content\_hi/products/Parallels-Utills.md  
\_content\_hi/products/Qemu-Utills.md  
\_content\_hi/products/Server-Factory-Additional-Components.md  
\_content\_hi/products/Server-Factory-Core-Framework.md  
\_content\_hi/products/Vasic-Digital-Reusable-Module-Suite.md  
\_content\_hi/products/VisionEngine.md  
\_content\_hi/products/docs\_chain.md  
\_content\_hi/products/task\_bridge.md  
\_content\_hi/sites/milosvasic-ru.md  
\_content\_hi/sites/vasic-digital.md  
\_content\_ja/docs/cover-letter.md  
\_content\_ja/docs/cv.md  
\_content\_ja/docs/portfolio.md  
\_content\_ja/products/Catalogizer.md  
\_content\_ja/products/Courses-Creator.md  
\_content\_ja/products/DocProcessor.md  
\_content\_ja/products/Helix-Flow.md  
\_content\_ja/products/HelixAgent.md  
\_content\_ja/products/HelixBuilder.md  
\_content\_ja/products/HelixCluster.md  
\_content\_ja/products/HelixCode.md  
\_content\_ja/products/HelixConstitution.md  
\_content\_ja/products/HelixGitpx.md  
\_content\_ja/products/HelixLLM.md  
\_content\_ja/products/HelixMemory.md  
\_content\_ja/products/HelixOTA.md  
\_content\_ja/products/HelixPlay.md  
\_content\_ja/products/HelixQA.md  
\_content\_ja/products/HelixSkills.md  
\_content\_ja/products/HelixSpecifier.md  
\_content\_ja/products/HelixTerminator.md  
\_content\_ja/products/HelixTrack.md  
\_content\_ja/products/HelixTranslate.md  
\_content\_ja/products/Herald.md  
\_content\_ja/products/LLMOrchestrator.md  
\_content\_ja/products/LLMProvider.md  
\_content\_ja/products/LLMsVerifier.md  
\_content\_ja/products/Mail-Server-Factory.md  
\_content\_ja/products/Parallels-Utills.md  
\_content\_ja/products/Qemu-Utills.md  
\_content\_ja/products/Server-Factory-Additional-Components.md  
\_content\_ja/products/Server-Factory-Core-Framework.md  
\_content\_ja/products/Vasic-Digital-Reusable-Module-Suite.md  
\_content\_ja/products/VisionEngine.md  
\_content\_ja/products/docs\_chain.md  
\_content\_ja/products/task\_bridge.md  
\_content\_ja/sites/milosvasic-ru.md  
\_content\_ja/sites/vasic-digital.md  
\_content\_kk/docs/cover-letter.md  
\_content\_kk/docs/cv.md  
\_content\_kk/docs/portfolio.md



\_content\_kk/products/Catalogizer.md  
\_content\_kk/products/Courses-Creator.md  
\_content\_kk/products/DocProcessor.md  
\_content\_kk/products/Helix-Flow.md  
\_content\_kk/products/HelixAgent.md  
\_content\_kk/products/HelixBuilder.md  
\_content\_kk/products/HelixCluster.md  
\_content\_kk/products/HelixCode.md  
\_content\_kk/products/HelixConstitution.md  
\_content\_kk/products/HelixGitpx.md  
\_content\_kk/products/HelixLLM.md  
\_content\_kk/products/HelixMemory.md  
\_content\_kk/products/HelixOTA.md  
\_content\_kk/products/HelixPlay.md  
\_content\_kk/products/HelixQA.md  
\_content\_kk/products/HelixSkills.md  
\_content\_kk/products/HelixSpecifier.md  
\_content\_kk/products/HelixTerminator.md  
\_content\_kk/products/HelixTrack.md  
\_content\_kk/products/HelixTranslate.md  
\_content\_kk/products/Herald.md  
\_content\_kk/products/LLMOrchestrator.md  
\_content\_kk/products/LLMProvider.md  
\_content\_kk/products/LLMsVerifier.md  
\_content\_kk/products/Mail-Server-Factory.md  
\_content\_kk/products/Parallels-Utills.md  
\_content\_kk/products/Qemu-Utills.md  
\_content\_kk/products/Server-Factory-Additional-Components.md  
\_content\_kk/products/Server-Factory-Core-Framework.md  
\_content\_kk/products/Vasic-Digital-Reusable-Module-Suite.md  
\_content\_kk/products/VisionEngine.md  
\_content\_kk/products/docs\_chain.md  
\_content\_kk/products/task\_bridge.md  
\_content\_kk/sites/milosvasic-ru.md  
\_content\_kk/sites/vasic-digital.md  
\_content\_ko/docs/cover-letter.md  
\_content\_ko/docs/cv.md

---

## 7. Internal consistency

Run on a read-only URI connection:

```
con = sqlite3.connect("file:<home>/local/share/lumen/
21bf1507a8925bcf/index.db?mode=ro",
                      uri=True, timeout=300)
con.execute("PRAGMA busy_timeout=300000")
for r in con.execute("PRAGMA integrity_check"): print(r[0])
```

**PRAGMA integrity\_check — verbatim output**

ok

**Supporting pragmas**

| Pragma                   | Result |
|--------------------------|--------|
| PRAGMA quick_check       | ok     |
| PRAGMA foreign_key_check | 0 rows |
| PRAGMA journal_mode      | wal    |
| PRAGMA page_size         | 4096   |
| PRAGMA page_count        | 24775  |
| PRAGMA freelist_count    | 0      |
| PRAGMA user_version      | 0      |

Clean, with no free pages — no sign of torn writes, rollback debris, or a half-vacuumed file. Executed at 2026-08-26T23:22:18+02:00 while lumen index was actively writing; the WAL was 8,285,352 bytes at the time and the check still returned ok.

**8. Conclusions and recommended action**

- 1. No vector in this index is corrupt.** All 30,009 were decoded and checked — 0 NaN, 0 Inf, 0 all-zero, 0 wrong-dimension, 0 bad norms — and the decoding was independently validated against the live model (cosine 0.98–0.99).
- 2. The ollama NaN fault could not corrupt the index by construction.** An HTTP 500 makes Lumen’s embed call return an error and the run aborts. No vector arrives, so none is stored. 140 log entries confirm every NaN hit terminated its run.
- 3. Aborted runs leave a partial but valid index.** files\_unchanged climbing 0 → 193 → 412 → 448 → 491 across six failed runs proves per-batch commits, each internally consistent.
- 4. The genuine damage is coverage.** 12.8% of the project is indexed. Semantic search over \_tests/, \_tools/, design-system/, docs/, scripts/, \_content\_sr|tr|zh/ will silently return nothing — the most dangerous failure mode here, because it looks like “no results” rather than an error.

5. **Do not purge or rebuild.** lumen purge would discard 30,009 verified-good vectors and restart a ~15-hour job. Let the running lumen index continue, or re-run it; it resumes from where it stopped.
6. **Watch for a recurrence.** ollama is healthy now, but the fault was intermittent for five weeks. Confirm completion with:

```
grep '"project": "<ext-volume-host>/Projects/vasic"' ~/.local/
share/lumen/debug.log \
| grep '"msg": "indexing complete"'
```

A genuine completion reports non-zero indexed\_files and chunks\_created. Re-verify coverage with:

```
SELECT COUNT(*) FROM files WHERE hash <> ''; -- target: ~3833
SELECT COUNT(*) FROM files WHERE hash = ''; -- target: 0
```

## One caveat worth flagging (inference, not measurement)

project\_meta.root\_hash advanced from e762a114... to 8f46dd1d... and last\_indexed\_at to 2026-08-26T21:25:19Z when the 23:11 run was **cancelled**, not completed. Lumen appears to persist these markers on run termination regardless of outcome. Per-file hashing still drives the real work — the very next plan (23:25:53) correctly found only 491 unchanged and 3,186 to add — so nothing was skipped in practice. But an incomplete index now carries a “fresh-looking” root hash, and if the tree stopped changing while coverage was partial, a fast-path “nothing to do” check on root\_hash could in principle report the index as up to date. I did not observe this happening; treat it as a risk to watch, not a confirmed defect.

---

## Appendix: reproducing this report

Every figure above comes from one of these, all read-only:

```
import sqlite3, numpy as np
DB = "file:<home>/~/.local/share/lumen/21bf1507a8925bcf/index.db?
mode=ro"
con = sqlite3.connect(DB, uri=True, timeout=120)
con.execute("PRAGMA busy_timeout=120000")

-- schema
SELECT type, name, tbl_name, sql FROM sqlite_master ORDER BY
type, name;
SELECT key, value FROM vec_chunks_info;
```

```

-- identity + counts
SELECT key, value FROM project_meta ORDER BY key;
SELECT COUNT(*) FROM files;
SELECT COUNT(*) FROM chunks;
SELECT COUNT(*) FROM vec_chunks_rowids;
SELECT COUNT(*) FROM files WHERE hash <> '';
SELECT COUNT(*) FROM files WHERE hash = '';

-- vector storage layout
SELECT c.chunk_id, c.size, length(c.validity), length(c.rowids),
       length(v.vectors)
FROM vec_chunks_chunks c
LEFT JOIN vec_chunks_vector_chunks00 v ON v.rowid = c.chunk_id
ORDER BY c.chunk_id;

-- vector decode inputs
SELECT id, chunk_id, chunk_offset FROM vec_chunks_rowids;
SELECT id, file_path, symbol, kind, start_line, end_line FROM
       chunks;
SELECT c.chunk_id, c.size, c.validity, c.rowids, v.vectors
FROM vec_chunks_chunks c
JOIN vec_chunks_vector_chunks00 v ON v.rowid = c.chunk_id
ORDER BY c.chunk_id;

-- referential integrity
SELECT COUNT(*) FROM chunks c LEFT JOIN vec_chunks_rowids r ON
       r.id = c.id WHERE r.id IS NULL;
SELECT COUNT(*) FROM vec_chunks_rowids r LEFT JOIN chunks c ON
       c.id = r.id WHERE c.id IS NULL;
SELECT COUNT(*) FROM vec_chunks_rowids WHERE chunk_id IS NULL OR
       chunk_offset IS NULL;
SELECT COUNT(*) FROM chunks c LEFT JOIN files f ON f.path =
       c.file_path WHERE f.path IS NULL;
SELECT COUNT(*) FROM chunks WHERE end_line < start_line;
SELECT COUNT(*) FROM chunks WHERE start_line < 0;
SELECT COUNT(*) FROM chunks WHERE symbol = '';

-- coverage
SELECT CASE WHEN instr(path, '/') > 0 THEN
       substr(path, 1, instr(path, '/') - 1) ELSE '<root file>' END
       AS top,
       COUNT(*) FROM files GROUP BY top ORDER BY 2 DESC;
SELECT f.path FROM files f LEFT JOIN chunks c ON c.file_path =
       f.path
WHERE c.id IS NULL ORDER BY f.path;
SELECT kind, COUNT(*) FROM chunks GROUP BY kind ORDER BY 2 DESC;

-- consistency
PRAGMA integrity_check;
PRAGMA quick_check;
PRAGMA foreign_key_check;

```

Vector decode, per block:

```
arr    = np.frombuffer(vectors, dtype='<f4').reshape(size, 768)
valid  = np.unpackbits(np.frombuffer(validity, dtype=np.uint8),
                        bitorder='little')[:size]
# check only slots present in vec_chunks_rowids:
# np.isnan(v).sum(), np.isinf(v).sum(), np.any(v),
# np.linalg.norm(v.astype(np.float64))
```

Chunk kind distribution, for reference:

| kind     | count  |
|----------|--------|
| section  | 29,692 |
| document | 31     |
| function | 14     |
| var      | 8      |

(Overwhelmingly Markdown sections, consistent with the alphabetically-early `_content*` and `_analysis` directories being the only ones reached so far. No source-code directory has been indexed yet.)